Table of Contents

Pass-Through: What Each Editor Copies, and What That Costs (2026-11-01)
- TL;DR
- 1. Method
  - 1.1 What pass-through means
  - 1.2 How it is measured
  - 1.3 The corpus
- 2. The package-level map
  - 2.1 Reading this table
  - 2.2 The full part matrix
  - 2.3 Two rows worth dwelling on
- 3. The architectures that produce pass-through
  - 3.1 Clone-and-override — Folio
  - 3.2 Byte-splice of untouched blocks — GenOffice and eigenpal
  - 3.3 The opaque-blob fallback — SuperDoc
  - 3.4 Manifest-gated passthrough — WordCraft
  - 3.5 The SDK round trip — Docxodus
  - 3.6 Why regeneration has none
- 4. What pass-through actually gives you
  - 4.1 But it is the weakest possible editing
  - 4.2 Bytes versus content
- 5. The failure modes pass-through hides
  - 5.1 The three kinds of dead weight
  - 5.2 The altChunk: a construct kept whole or dropped whole
  - 5.3 The chart's two companions
  - 5.4 The dangling reference: office-open and Docen
  - 5.5 The good news: content-types are coherent everywhere
  - 5.6 A note on dgm:relIds
- 6. Per-editor pass-through
  - 6.1 SuperDoc — 27 parts, the whole payload
  - 6.2 Folio — 34 parts, the author's compressed bytes
  - 6.3 GenOffice — 20 parts, byte-exact at the block level
  - 6.4 eigenpal — 5 parts, but the content of all of them
  - 6.5 office-open and Docen — 9 parts each, and five dangling references
  - 6.6 WordCraft — 16 parts, and the chart's two companions

6.7 Docxodus — 25 parts byte-identical, and nothing changed at all
6.8 Fidus Writer, canvas-editor, docs.plus, Plate.js — zero
7. What this means
Reproducing this
Gaps and honest limits

Pass-Through: What Each Editor Copies, and What That Costs (2026-11-01)

A companion to *Open-Source DOCX Editors, Re-verified Against Code and Measured by Round-Trip (2026-11-01)*, and the second half of what used to be *Report A*. The first report asked how faithfully each editor preserved a document and answered with a number. *Serialization by Namespace* opened the question up vocabulary by vocabulary. This one asks the complementary question: **of the bytes that came out, how many did the editor never even look at?**

That sounds like a dull question. It is the opposite. Pass-through is the **strongest possible fidelity and the weakest possible editing**, and the two facts are the same fact: an editor that copies a part byte-for-byte cannot change any of it. So the part map does not measure what an editor preserved — it measures **what an editor declined to understand**.

TL;DR

- **Folio copies 34 of the corpus's 35 parts byte-for-byte and still scores 98.5%.** The reason is that the one part it rebuilds — `word/document.xml` — is where almost all of the interesting markup lives. **Pass-through is a measure of what the editor ignored, not of what it preserved.**
- **Docxodus is the only tool in the study with nothing changed and nothing dropped: 25 parts byte-identical, 10 re-serialised, 0 changed, 0 dropped, 0 added, all 35 parts out.** It gets there without a policy at all — it is built on the Microsoft Open XML SDK, opens the package with `WordprocessingDocument.Open(stream, isEditable: true)` and saves the same object graph back out, so parts it has no schema for are carried through untouched and parts it owns are re-printed. **Folio copies more bytes; Docxodus changes less.**
- **Folio's pass-through is of the raw compressed bytes, not the decompressed XML.** `docx-core/src/zip/archive.ts:388-405` writes the original deflate stream, so the CRC and the compressed size are preserved exactly. No editor in this study returns an author's bytes more faithfully.
- **Five architectures produce pass-through, and they are not equally faithful.** Clone-and-override (Folio) copies compressed bytes. Byte-splice (GenOffice, eigenpal) copies *byte ranges of the decompressed XML*. Opaque-blob (SuperDoc) copies whole ZIP entries it never modelled. Manifest-gated (WordCraft) copies payloads only when a manifest and a passthrough flag agree. **SDK round trip (Docxodus)** lets the Open XML SDK own the parts it models and pass the rest through.
- **Four editors have zero pass-through.** Fidus Writer, canvas-editor, docs.plus and Plate.js all regenerate every part. Plate.js is the extreme: **zero byte-identical parts, 24 parts dropped, nothing added, 11 parts out** — the smallest package in the trial.
- **Pass-through has a hidden failure mode that no part-level checker catches: the orphan.** When an editor keeps the payload but drops the element that pointed at it, the package still contains the bytes and nothing references them. **That failure mode did not happen with the `altChunk` — it was kept whole by seven editors and dropped whole by six — but it did happen with the chart, and in a worse form (§5.4).**
- **A worse variant exists, and two editors have it: the dangling reference.** office-open and Docen rewrite the body's relationship ids for the chart and the SmartArt to `rId17`–`rId21` **and do not write those relationships**. The chart part is present, the original relationship is present and resolves, and the body points at an id that does not exist. **Five dangling references each, and a chart that never draws** — in a package that is otherwise schema-clean and free of dangling *parts*. **Docxodus has none**, and neither does anything else in the study.
- **Content-types declarations are coherent in every single output.** No editor leaves a part declared in `[Content_Types].xml` that is absent from the package. That is a genuine piece of good news, and it is worth stating because it is the failure mode one would expect here.

---

1. Method

1.1 What pass-through means

Pass-through is the case where the editor never looked at the bytes. It copied them because it had no reason to touch them.

This is the strongest fidelity available — the author's bytes come back exactly as they went in — and the cheapest: **the cost of understanding a part is only worth paying if you intend to change it.** An editor that models a chart can restyle it, re-scale it, re-point it at new data. An editor that copies the chart cannot do any of those things, and the user gets a document they can read but not really edit.

Three distinctions matter throughout:

- **Pass-through is not serialization.** An editor may *parse* a part and re-emit an identical tree; that is serialization with a null result, not pass-through. The measurement separates them.
- **Pass-through is not preservation of content.** A byte-identical part preserves the part. Whether the document still *uses* that part is a separate question — and §5.2 is about the editors where it does not.
- **Pass-through at the part level is not pass-through at the element level.** An editor can copy a part and change one element inside it.

1.2 How it is measured

`build/part_map.py` compares every part of every output against the input and classifies it:

Code	Meaning
P	**Pass-through** — byte-identical to the input
w	**Rewritten** — the bytes differ but the XML tree is identical (a different serialiser, the same content)
C	**Changed** — the tree differs
D	**Dropped** — the part is not in the output
A	**Added** — the part is in the output and not in the input
.	not present on either side

XML comparison is **prefix-independent**: a writer that renames `w14:` to `wordml:` is not mistaken for a writer that changes content. Binary parts are compared byte-for-byte.

Three further audits in this report go past the part map:

Tool	What it adds
`build/orphan_audit.py`	Finds payload parts that survive but are referenced by nothing — and distinguishes *unreferenced* (no relationship points at them) from *body-orphaned* (a relationship points at them, but the body no longer uses that relationship id).
`build/rel_forward.py`	Resolves every `r:id` / `r:embed` / `r:dm` / `r:lo` / `r:qs` / `r:cs` **from the body forward** into the part's `.rels` file, and reports the ids that do not resolve.
`build/ct_coherence.py`	Checks that every part declared in `[Content_Types].xml` is present in the package, and that every part has a content type.

1.3 The corpus

The same 35-part adversarial corpus (`fixture/complex.docx`) used throughout the study, corrected so Microsoft Word 16.115 opens it: a chart, four SmartArt parts, an OLE object with a real embedded `.docx`, an embedded workbook, an image, two tables, tracked changes, content controls, fields, bookmarks, notes, comments, OMML math, `mc:AlternateContent`, a `w:altChunk`, `w14:*` extensions, `w16du:dateUtc`, and three sections including a landscape one.

2. The package-level map

P byte-identical pass-through · W rewritten, same XML tree · C changed · D dropped · A added

Editor	Pass-through	Rewritten	Changed	Dropped	Added	Parts out
Folio	**34**	0	1	0	0	35
SuperDoc	**27**	1	7	0	0	35
Docxodus	**25**	**10**	**0**	**0**	**0**	**35**
Apryse WebViewer	9	8	13	5	27	57
GenOffice	20	13	2	0	0	35
WordCraft	16	0	16	3	4	36
office-open	9	10	16	0	0	35
Docen	9	9	17	0	4	39
eigenpal	5	16	14	0	0	35
LibreOffice (baseline)	2	3	20	10	18	43
Fidus Writer	0	0	11	24	3	14
canvas-editor + plugin	0	0	14	21	6	20
docs.plus	0	0	11	24	2	13
Plate.js	**0**	**0**	**11**	**24**	**0**	**11**

2.1 Reading this table

Read it with the first report's scores and it stops being a ranking. Folio copies more of the input byte-for-byte than any editor, and still scores 97.5% — because the part it rebuilds, `word/document.xml`, is where almost all of the interesting markup lives. Meanwhile Plate.js copies **nothing** and scores 20.6%. The pass-through count and the fidelity score are measuring different things, and the correlation between them is close to zero.

Four editors have zero pass-through, and they fail in three different ways. Fidus Writer, canvas-editor, docs.plus and Plate.js all regenerate every part from a model.

- **Fidus Writer, canvas-editor and docs.plus all drop 24 of 35 parts** — the same 24, essentially: the chart, the four SmartArt parts, the embeddings, the image, the headers and footers, the notes, the comments, the custom XML, the `altChunk`. What differs is what they put back.
- **Plate.js is the extreme.** It drops the same 24 and **adds nothing at all**: 0 added, 11 parts out. Its output is the smallest regenerated package in the trial — 8,010 bytes on the second pass, against the corpus's 42,964. The reason is architectural: the DOCX pipeline is **DOCX → HTML → editor model → static HTML → DOCX**, and HTML cannot carry a header, a footer, a footnote, an endnote, a comment, a glossary part, a customXml part, an `altChunk` part, a chart, a SmartArt diagram, an OLE object, an embedded workbook or an image reference.

The `W` column is the most interesting and the most misread. GenOffice has 13 parts that are *rewritten but semantically identical* — same tree, different bytes — and only 2 that genuinely changed. eigenpal has 16. These are **not** losses. They are the cost of a different serialiser: attribute order, self-closing form, entity form and namespace-prefix binding all differ, and the content does not. An editor that scored only "byte-identical" would look like it had rewritten 13 parts when in fact it had rewritten two and re-printed eleven.

SuperDoc's 27 pass-through parts are the entire binary payload. The chart, the four SmartArt parts, the OLE embedding, the workbook, the image, `customXml`, `webSettings`, `fontTable`, `theme` and `afchunk.html` are all byte-identical — because SuperDoc's engine does not model them and returns the raw ZIP entries.

Docen and WordCraft are the only editors that *add* parts that were not in the input (4 each). Docen adds the three Word 2016 comment-extension parts (`commentsExtended.xml`, `commentsExtensible.xml`, `commentsIds.xml`) and `docProps/custom.xml`. WordCraft adds `docProps/custom.xml` and the header/footer `.rels` parts.

2.2 The full part matrix

The matrix is printed as four tables — two editor halves × two part groups — because fourteen editor columns do not survive a landscape page. **Part names are printed without the `word/` prefix**, with sub-directories compacted (`embeddings/` to `embed/`, `diagrams/` to `dgm/`, `charts/` to `chart/`) and `Microsoft_Excel_Worksheet.xlsx` shortened to `Excel_Worksheet.xlsx`; parts outside `word/` keep their full path, so `_rels/.rels` and `customXml/item1.xml` are unambiguous. **Editor names are shortened in the header only** — `Fidus Writer` to `Fidus`, `canvas-editor + plugin` to `canvas + plugin`, `LibreOffice (baseline)` to `LibreOffice`.

WordprocessingML and package parts

Package part	Plate.js	Docxodus	SuperDoc	Folio	GenOffice	eigenpal
`[Content_Types].xml`	C	P	P	P	P	P
`_rels/.rels`	C	P	P	P	P	w
`customXml/item1.rels`	D	P	P	P	P	w
`customXml/item1.xml`	D	P	P	P	P	w
`customXml/itemProps1.xml`	D	P	P	P	P	w
`docProps/app.xml`	D	P	P	P	P	w
`docProps/core.xml`	C	P	P	P	C	w
`_rels/document.xml.rels`	C	P	P	P	P	w
`chart/_rels/chart1.xml.rels`	D	P	P	P	P	w
`comments.xml`	D	w	w	P	w	C
`document.xml`	C	w	C	C	C	C
`endnotes.xml`	D	w	C	P	w	C
`fontTable.xml`	C	P	P	P	w	C
`footer1.xml`	D	w	C	P	w	C
`footer2.xml`	D	w	C	P	w	C
`footnotes.xml`	D	w	C	P	w	C
`glossary/document.xml`	D	P	P	P	w	C
`header1.xml`	D	w	C	P	w	C
`header2.xml`	D	w	C	P	w	C
`numbering.xml`	C	P	P	P	w	C
`settings.xml`	C	w	P	P	w	C
`styles.xml`	C	w	P	P	w	C
`theme/theme1.xml`	C	P	P	P	P	w

Package part	Plate.js	Docxodus	SuperDoc	Folio	GenOffice	eigenpal
webSettings.xml	C	P	P	P	w	C

WordprocessingML and package parts (continued)

Package part	office-open	Docen	WordCraft	Fidus	canvas + plugin	docs.plus	LibreOffice
[Content_Types].xml	w	C	C	C	C	C	C
_rels/.rels	w	w	P	C	C	C	C
customXml/item1.rels	P	P	P	D	D	D	C
customXml/item1.xml	P	P	P	D	D	D	w
customXml/itemProps1.xml	P	P	P	D	D	D	w
docProps/app.xml	C	C	C	C	C	D	C
docProps/core.xml	w	w	C	C	C	C	C
_rels/document.xml.rels	C	C	C	C	C	C	C
chart/_rels/chart1.xml.rels	w	w	P	D	D	D	D
comments.xml	C	C	C	D	C	D	C
document.xml	C	C	C	C	C	C	C
endnotes.xml	C	C	C	D	D	D	C
fontTable.xml	C	C	P	C	C	C	C
footer1.xml	C	C	C	D	C	D	C
footer2.xml	C	C	C	D	D	D	C
footnotes.xml	C	C	C	C	C	D	C
glossary/document.xml	C	C	P	D	D	D	w
header1.xml	C	C	C	D	C	D	C
header2.xml	C	C	C	D	D	D	C
numbering.xml	C	C	C	C	C	C	C
settings.xml	C	C	C	C	C	C	C
styles.xml	C	C	C	C	C	C	C
theme/theme1.xml	w	w	C	D	D	C	C
webSettings.xml	C	C	P	D	D	C	D

Binary payload and opaque parts

Package part	Plate.js	Docxodus	SuperDoc	Folio	GenOffice	eigenpal
chart/chart1.xml	D	P	P	P	P	w

Package part	Plate.js	Docxodus	SuperDoc	Folio	GenOffice	eigenpal
`chart/style1.xml`	D	P	P	P	P	w
`chart/colors1.xml`	D	P	P	P	P	w
`dgm/data1.xml`	D	P	P	P	P	w
`dgm/layout1.xml`	D	P	P	P	P	w
`dgm/quickStyle1.xml`	D	P	P	P	P	w
`dgm/colors1.xml`	D	P	P	P	P	w
`embed/embedded1.docx`	D	P	P	P	P	P
`embed/Excel_Worksheet.xlsx`	D	P	P	P	P	P
`media/image1.png`	D	P	P	P	P	P
`afchunk.html`	D	P	P	P	P	P

Binary payload and opaque parts (continued)

Package part	office-open	Docen	WordCraft	Fidus	canvas + plugin	docs.plus	LibreOffice
`chart/chart1.xml`	w	w	P	D	D	D	C
`chart/style1.xml`	P	P	D	D	D	D	D
`chart/colors1.xml`	P	P	D	D	D	D	D
`dgm/data1.xml`	w	w	P	D	D	D	D
`dgm/layout1.xml`	w	w	P	D	D	D	D
`dgm/quickStyle1.xml`	w	w	P	D	D	D	D
`dgm/colors1.xml`	w	w	P	D	D	D	D
`embed/embedded1.docx`	P	P	P	D	D	D	D
`embed/Excel_Worksheet.xlsx`	P	P	P	D	D	D	P
`media/image1.png`	P	P	P	D	D	D	P
`afchunk.html`	P	P	D	D	D	D	D

2.3 Two rows worth dwelling on

`afchunk.html` is pass-through for seven participants and dropped by six. The seven that keep it are SuperDoc, Folio, GenOffice, eigenpal, office-open, Docen and **Docxodus**. The six that drop it are WordCraft, Fidus, canvas-editor, docs.plus, Plate.js and LibreOffice.

`chart/style1.xml` and `chart/colors1.xml` are dropped by WordCraft and by LibreOffice — the two outputs that keep the chart itself while losing its companions. WordCraft copies the chart part byte-for-byte and loses the chart's style and colour companions. §5.3 explains why that is worse than it looks.

3. The architectures that produce pass-through

The part map has **five** distinct causes, and they differ in exactly what the author's bytes are entitled to.

3.1 Clone-and-override — Folio

Folio's policy is stated in its own source (`packages/core/src/docx/packageParts.ts:4-27`):

> Folio models a handful of those parts — the body, the styles, the notes — and rewrites them on save. Everything else belongs to the author, and a repack hands it back byte for byte.

The mechanism is a clone of the source ZIP with only the modelled parts overridden (`rezip.ts:1864`, `:1877`, `:1644`), and the copy is **of the compressed bytes**: `docx-core/src/zip/archive.ts:388-405` writes the raw deflate stream, so the CRC and the compressed size are preserved exactly. Parts that are never decoded stay that way (`unzip.ts:951-960`).

What Folio rebuilds: `word/document.xml` in full (`rezip.ts:1999`); every header and footer (`:3141-3143`); footnotes and endnotes, but by **splicing** rather than rebuilding (`:3229-3236`); `styles.xml`, by patching (`:3722-3739`); `comments.xml`, preserved untouched when unchanged (`:473-477`).

That is why Folio's part map has 34 pass-through parts: the corpus's headers, footers, notes, comments, chart, diagrams, embeddings, media, customXml, theme, fontTable and webSettings are all *the author's bytes, returned unchanged*.

3.2 Byte-splice of untouched blocks — GenOffice and eigenpal

Both keep the source document's XML as a sequence of blocks and re-serialize only the blocks they changed.

- **GenOffice** parses with `packages/docx-engine/src/parse.ts:360` (`parseDocx`), converts to its editor model at `apps/docs/src/renderer/editor/convert.ts:139` (`blocksToPmDoc`), plans the save at `:2196` (`pmDocToSavePlan`), and applies it in `packages/docx-engine/src/patch.ts:723` (`saveDocx`). The patcher splices the untouched blocks back **byte-exactly** (`patch.ts:1644-1648`) and, for tables, falls back to copying the source part when its signature does not match the current model (`patch.ts:2134`). The result is re-zipped, not copied as compressed bytes.
- **eigenpal** reads the package with `packages/core/src/store/package/ooxml-package.ts:418` (`readOoxmlPackage`) and writes it at `:624` (`writeOoxmlPackage`). Its writer **starts from the original entry bytes and overwrites only the parts it holds as trees**, so binary and unmodelled XML parts are byte-preserved while the modelled ones are re-serialized in a normalized form.

3.3 The opaque-blob fallback — SuperDoc

SuperDoc's engine is closed-source, but its contract is visible: `SuperDoc.ts:4016-4023` contains a byte-identity fallback that returns the original blob untouched, and the engine's public type for parts it does not model is `RawDocxPart` with `kind: 'added' | 'removed' | 'changed'` and `source: 'opaque-zip-entry'` (`docx-engine.d.ts:24-54`). The engine also exposes three export modes — `"review-preserving" | "final" | "original"` (`docx-engine.d.ts:73`) — which is the correct shape for a round-trip tool, because "leave it exactly as it was" has to be an explicit choice.

3.4 Manifest-gated passthrough — WordCraft

WordCraft is a hybrid with a twist: `crates/docx/src/package.rs:470-482` rebuilds the ZIP and **re-compresses everything** (deflate level 6; PNG, JPEG, GIF and WEBP are stored), so nothing is byte-identical *at the ZIP level* — yet 16 of its output parts are byte-identical *as content*, because binary payloads are copied through a manifest-gated passthrough (`crates/docx/src/read/embed.rs:182` `keep_embedded`; `crates/docx/src/write/embed.rs:297` `copy_tree`; `crates/docx/src/write/embed.rs:186-189` `available()` requires both the manifest and the passthrough).

3.5 The SDK round trip — Docxodus

Docxodus is the only tool in the study that does not have a *policy* about pass-through, because it never takes the package apart. It is built on the Microsoft **Open XML SDK**, and the session opens the document the way the SDK does (`Docxodus/DocxSession.cs:166`):

```
_doc = WordprocessingDocument.Open(_stream, isEditable: true);
```

The SDK has a **schema for the whole of WordprocessingML**, so it can parse every part it knows; for every part it does not know it keeps the entry as it found it; and inside a part it does know, an element or attribute from a namespace it has no schema for is still carried. The last of those is worth being precise about, because the mechanism is *not* the SDK's unknown-element machinery: **Docxodus never manipulates the SDK's typed DOM at all.** It works on LINQ-to-XML `XDocument` trees (`Docxodus/PtOpenXmlUtil.cs:141`), and `OpenXmlUnknownElement` appears **nowhere** in the repository. An element the SDK has never heard of is simply a node in a tree, and nodes in a tree are carried. The companion report *Serialization by Namespace* §3.12 works this through in full. The write path is the same object graph saved back out (`DocxSession.cs:819` `Save(bool persistAnchorIds)` → `_doc.Save()`), followed by one Docxodus-specific step — `ZipPackageOutputNormalizer.Normalize(...)` (`Docxodus/ZipPackageOutputNormalizer.cs:46`), whose own header states the rule:

> *"The package payload is copied entry-for-entry; only ZIP compression and extraction metadata are normalized."*

The source is equally explicit about what that does and does not imply (`DocxSession.cs:871`):

> *"The clone re-serializes the package-level relationship and content-type files, so the ZIP is not byte-identical to a save of a document Word wrote; part payloads are."*

And the measurement is exactly that. 25 parts are byte-identical. Ten are byte-different and tree-identical — `word/document.xml`, `comments.xml`, `endnotes.xml`, `footnotes.xml`, `header1/2.xml`, `footer1/2.xml`, `settings.xml` and `styles.xml` — because the SDK owns those DOMs and re-serialises them. **Nothing is changed, nothing is dropped, nothing is added.**

One more piece of Docxodus's architecture matters for the rest of the study: it opens every package through a **repair funnel** first (`PackageOpenRepair`): *"Package defects Word opens but the Open XML SDK rejects, repaired once on the way in so every front door (session, comparison, conversion) opens the package and every save writes it back (issue #1384)."* What it repairs is a relationship to a part with no content type or no ZIP entry, a part reached by a known relationship but typed only as generic XML, and two relationships sharing an Id. **That is a different defect class from the twelve in Report B**, and the measurement there shows Docxodus does *not* repair those.

3.6 Why regeneration has none

- **office-open** (`packages/docx/src/parse.ts:501` → `packages/docx/src/generate.ts:71`) and **Docen** (which layers a Tiptap representation on top at `packages/docx/src/converters/docx.ts:1337`/`:1415`).
- **canvas-editor + plugin** rebuilds the package with the `docx` npm package (`packages/docx/src/docx/exportDocx.ts:796`), which brings its own default part set — styles, numbering, theme, settings, fontTable, webSettings, headers, footers — and writes the body from the canvas's element list.
- **docs.plus** runs DOCX → HTML (`mammoth`) → Tiptap JSON → HTML → DOCX (`@turbodocx/html-to-docx`).
- **Fidus Writer** does something categorically different: its exporter takes a **template URL** (`src/exporter/docx/index.ts:34`) and fills that template's placeholders (`src/exporter/docx/render.ts:746-834`, `:982-1021`; the placeholder vocabulary is defined in `src/exporter/template/tags.ts:18-101` — `{title}`, `{@<part-id>}` for rich-text and table parts, plus `{@bibliography}`, `{@copyright}`, `{@licenses}`). The output package is the **template's**, not the input's. The exported `word/styles.xml`, `word/numbering.xml` and `word/settings.xml` are the template's, which is why they show as C while every input-specific part is D.
- **Plate.js** regenerates through an HTML middle: `importDocx` runs **mammoth** into HTML and decodes that through the plugin set's own `formats.html` mappings (`import/lib/importDocx.ts:2427`), and `exportDocx` walks the model back out to **static HTML** and then to the DOCX writer (`export/lib/exportDocx.tsx:646`; `export/lib/internal/html-to-docx.ts`).

---

4. What pass-through actually gives you

The strongest fidelity available, and it is not close.

Folio returns the author's `word/charts/chart1.xml` with the same CRC and the same compressed size. GenOffice returns the author's paragraph bytes as a literal slice of the source document. SuperDoc returns the author's ZIP entries untouched. **A user who opens the corpus in Folio, edits one paragraph, and saves gets back 34 parts that are bit-for-bit what they started with** — including the chart's style and colour companions, the four SmartArt parts, the embedded workbook, the image, and the `altChunk` HTML.

That is the ideal a round-trip tool should aim at, and only five participants in this study come close — Folio, SuperDoc, Docxodus, GenOffice and eigenpal.

4.1 But it is the weakest possible editing

The same property, seen from the other side: **an editor that copies a part byte-for-byte cannot round-trip a change to any of it.** Folio can change the body, the headers, the footers, the notes and the styles — and nothing else. Its 34 pass-through parts are 34 parts in which a user cannot restyle a chart, re-point an image, replace an embedded workbook, or change a header that Folio did not rebuild.

The general rule the part map makes visible: **the parts an editor does not model are the parts a user will not be able to change.**

4.2 Bytes versus content

Folio's compressed-bytes copy and GenOffice's decompressed-byte splice are both "byte-exact" and they are not the same thing.

Folio's copy preserves the **compressed representation** — the author's deflate stream, the author's CRC. GenOffice's splice preserves the **decompressed byte range** but the package is re-zipped, so the compression differs. Both are excellent fidelity; the second is one step further from the original file, and it is the only one of the two that can be edited at the block level.

The first report's part map compares **decompressed content** for XML parts and **raw bytes** for binary parts, so for Folio both agree. The compressed-stream detail comes from Folio's own source and is stated here as a property of the implementation rather than a separate measurement.

---

5. The failure modes pass-through hides

This section is the reason the report exists. A part map tells you what came out. It does not tell you whether **what came out is still connected to anything**.

5.1 The three kinds of dead weight

Kind	Definition	Detected by
Dangling part	A `.rels` entry points at a part that is not in the package.	`orphan_audit.py`
Unreferenced part	The part is in the package and no relationship points at it.	`orphan_audit.py`
Body-orphaned part	The part is in the package and a relationship points at it, but **the body no longer uses that relationship id**.	`orphan_audit.py`
Dangling reference	The body uses a relationship id that its `.rels` file **does not define**.	`rel_forward.py`

The last two are the dangerous ones, because both leave a package that passes every structural check.

5.2 The `altChunk`: a construct kept whole or dropped whole

The corpus's `w:altChunk` element points at `word/afchunk.html` through the relationship `rIdAltChunk`:

```
<Relationship Id="rIdAltChunk" Type".../relationships/aFChunk" Target="afchunk.html"/>
...
<w:altChunk r:id="rIdAltChunk"/>
```

This is the natural test case for the orphan failure mode, because the construct is split across two places: an **element in the body** and a **part in the package**. An editor that copies the part but drops the element leaves the payload in the file with nothing pointing at it.

Measured, it does not happen. Not once.

Editor	`w:altChunk` element in the body	`word/afchunk.html` part	Half-preserved?
corpus	yes	yes	—
SuperDoc	yes	yes (pass-through)	no
Folio	yes	yes (pass-through)	no
GenOffice	yes	yes (pass-through)	no
Docxodus	**yes**	**yes (pass-through)**	**no**
eigenpal	yes	yes (pass-through)	no
office-open	yes	yes (pass-through)	no
Docen	yes	yes (pass-through)	no
WordCraft	no	no	no
Fidus Writer	no	no	no
canvas-editor	no	no	no
docs.plus	no	no	no
Plate.js	no	no	no
LibreOffice	no	no	no

Seven participants keep both halves and six keep neither. The `altChunk` is a construct the modelling decision either understands or does not, and the decision is applied consistently to the body element and to the payload. The part-level map records the HTML part as pass-through for seven participants, and in every one of those seven the element that uses it is still there.

That is worth stating as a **positive** result, because it is the outcome a naive reading of the part map would not predict: a reader seeing "`word/afchunk.html` is byte-identical in seven outputs" would reasonably assume the chunk was **orphaned** — kept as ballast, unreferenced. It is not. **Pass-through at the part level did not produce an orphan here**, because the same modelling decision governs both halves.

§5.4 is where it does go wrong — and there it goes wrong in a way no part-level checker can see.

5.3 The chart's two companions

`word/charts/style1.xml` and `word/charts/colors1.xml` carry the chart's **appearance**. The chart part holds the data; these hold how it looks.

In the corpus, neither is the target of any relationship — they are bound to the chart by part-name convention and by their `[Content_Types].xml` declaration, which is how the format actually works for these two parts. So they are "unreferenced" in the corpus too, and they stay unreferenced in every output that keeps them.

WordCraft and LibreOffice are the two outputs that keep `chart1.xml` and drop both companions. WordCraft's output therefore contains a chart with the author's data and the default theme instead of the author's design; LibreOffice's, rebuilt from its own model, loses the same two parts. The part map records the drop as D; nothing in the package says the chart looks wrong.

`build/ct_coherence.py` confirms that both also removed the two `[Content_Types].xml` overrides, so the packages are internally consistent. **The loss is semantic and completely invisible to a structural checker.**

5.4 The dangling reference: office-open and Docen

This is the most serious finding in the report, and it is invisible to every checker that only looks at the relationship graph in one direction.

The corpus's chart and SmartArt are referenced from the body:

```
<w:drawing>… <c:chart r:id="rIdChart"/> …</w:drawing>
<w:drawing>… <dgm:relIds r:dm="rIdDgmData" r:lo="rIdDgmLayout"
                         r:qs="rIdDgmQuickStyle" r:cs="rIdDgmColors"/> …</w:drawing>
```

SuperDoc, Folio, GenOffice and eigenpal keep those ids exactly. **office-open and Docen do not.** They rewrite the body's ids to a fresh numbering scheme and do not write the corresponding relationships:

Editor	Chart id in body	Diagram ids in body	Defined in `word/_rels/document.xml.rels`?
corpus	`rIdChart`	`rIdDgmData`, `rIdDgmLayout`, `rIdDgmQuickStyle`, `rIdDgmColors`	yes
SuperDoc	`rIdChart`	same four	yes
Folio	`rIdChart`	same four	yes
GenOffice	`rIdChart`	same four	yes
Docxodus	**`rIdChart`**	**same four**	**yes**
eigenpal	`rIdChart`	same four	yes
office-open	**`rId17`**	**`rId18`, `rId19`, `rId20`, `rId21`**	**no — none of the five exist**
Docen	**`rId21`**	**`rId22`, `rId23`, `rId24`, `rId25`**	**no — none of the five exist**

`build/rel_forward.py` resolves every relationship id used anywhere in every XML part against that part's `.rels` file, and finds:

Editor	Dangling references	Detail
corpus, SuperDoc, Folio, GenOffice, eigenpal, **Docxodus**, WordCraft, Fidus, canvas, docs.plus, Plate.js, LibreOffice	**0**	—
office-open	**5**	`document.xml` uses `id=rId17`, `dm=rId18`, `lo=rId19`, `qs=rId20`, `cs=rId21` — none defined
Docen	**5**	`document.xml` uses `id=rId21`, `dm=rId22`, `lo=rId23`, `qs=rId24`, `cs=rId25` — none defined

Meanwhile the **original** relationships are still there and still resolve:

```
<Relationship Id="rIdChart" Type=".../relationships/chart" Target="charts/chart1.xml"/>
<Relationship Id="rIdDgmData" Type=".../relationships/diagramData" Target="diagrams/data1.xml"/>
...
```

So the package contains: the chart part, the four SmartArt parts, the correct relationships pointing at them — **and a body that points at five relationship ids which do not exist.** The chart is present and unreachable. The SmartArt is present and unreachable. **The `w:drawing` elements survive; the links inside them do not.**

This is precisely why office-open scores 100% on text retention at the XML level and still loses 12 rendered lines. The text is all still in `word/document.xml`; the chart is still there as an element; but the relationship that binds the chart to the package no longer resolves, so nothing draws it. **A checker that reads XML sees the chart. A reader looking at a page does not.**

5.5 The good news: content-types are coherent everywhere

`build/ct_coherence.py` checks every output for parts declared in `[Content_Types].xml` that are not in the package, and for parts with no content type at all.

Editor	Parts	Content-type overrides	Declared but absent	Untyped parts
corpus	35	25	—	—
SuperDoc	35	25	—	—
Folio	35	25	—	—
GenOffice	35	25	—	—
Docxodus	**35**	**25**	**—**	**—**
eigenpal	35	25	—	—
office-open	35	25	—	—
Docen	39	29	—	—
WordCraft	36	30	—	—
Fidus	14	11	—	—
canvas	20	12	—	—
docs.plus	13	10	—	—
Plate.js	11	10	—	—
LibreOffice	43	37	—	—

Not one editor leaves a dangling content-type declaration. Every editor that drops a part also removes its declaration. This is worth stating explicitly because it is the most obvious way a lossy pipeline could produce an invalid package, and none of these twelve does it.

5.6 A note on `dgm:relIds`

The SmartArt's four-part linkage is a good illustration of why the dangling-reference check matters. A SmartArt diagram is **four coordinated parts plus four relationship ids in the body**. Losing one id leaves three that resolve and one that does not — a state that is:

- invisible to a part-level comparison (all four parts are present),
- invisible to a schema validator (the attribute is well-formed),
- invisible to a content-types check,
- and **fatal to the rendering**.

No editor in this study is in that partial state — office-open and Docen lose all four together, and the other editors lose none. But the state is reachable, and it is the kind of thing a package-level fidelity score cannot see.

---

6. Per-editor pass-through

6.1 SuperDoc — 27 parts, the whole payload

Pass-through. 27 of 35 parts are byte-identical. The binary payload — chart, four SmartArt parts, the OLE embedding, the workbook, the image — is entirely byte-identical, as are `customXml`, `webSettings`, `fontTable`, `theme` and `afchunk.html`.

The one part it rewrites without changing is `word/comments.xml` (`w`) — same tree, different bytes.

The seven it changes are `word/document.xml`, `docProps/core.xml`, `word/endnotes.xml`, both footers, `word/footnotes.xml` and both headers — that is, the parts SuperDoc actually models.

What it does not do. It does not orphan the payload. Its body's relationship ids are the corpus's own, and every one resolves.

6.2 Folio — 34 parts, the author's compressed bytes

Pass-through. 34 of 35 parts, as raw compressed bytes with the original CRC and compressed size.

The one part it rebuilds is `word/document.xml` — and that single part is where nearly all of the corpus's interesting markup lives, which is why 34 parts of pass-through and a 98.5% score coexist.

What it does not do. It orphans nothing: its body's relationship ids are the corpus's own, and every one resolves (§5.4).

6.3 GenOffice — 20 parts, byte-exact at the block level

Pass-through. 20 parts, including the entire binary payload and `afchunk.html`.

The 13 rewritten-but-identical parts are the cost of re-serializing `document.xml`'s siblings — same tree, different bytes.

The two genuinely changed parts are `word/document.xml` and `docProps/core.xml`.

What it does not do. It orphans nothing; its relationship ids all resolve.

6.4 eigenpal — 5 parts, but the content of all of them

Pass-through. Only 5 parts are byte-identical — but the *content* of the binary payload is byte-identical, and the 16 "rewritten" parts are re-serialized to the same tree.

This row is the clearest example of why the `P` count alone is misleading. eigenpal looks like the worst pass-through performer **by the `P` count** — 5, against Folio's 34 and Docxodus's 25 — and it is the second best on every other measure in the study. Its writer starts from the original entry bytes and overwrites only the parts it holds as trees (`ooxml-package.ts:418`, `:624`), so **nothing is lost; it is only re-printed**.

What it does not do. It orphans nothing; its relationship ids all resolve.

6.5 office-open and Docen — 9 parts each, and five dangling references

Pass-through. 9 each.

What they do that nobody else does. They rewrite the body's relationship ids for the chart and the SmartArt to ids that their own `.rels` files do not define (§5.4). The chart part, the four SmartArt parts and the original relationships are all still there. **The links in the body are not.**

This is the one place in the study where a pass-through-oriented editor produced a package that is *structurally perfect and functionally broken at the same time*: no part is missing, no content-type declaration dangles, the XML is schema-clean — and the chart does not draw.

6.6 WordCraft — 16 parts, and the chart's two companions

Pass-through. 16 parts by content, through a manifest-gated passthrough, even though the ZIP itself is rebuilt and re-compressed.

The three it drops are `word/charts/style1.xml`, `word/charts/colors1.xml` and `word/afchunk.html`.

What it does not do. It has no dangling references, and it orphans nothing — because it drops the parts it cannot use and removes their declarations.

6.7 Docxodus — 25 parts byte-identical, and nothing changed at all

Pass-through. 25 of 35 parts are byte-identical and the other ten are re-serialised to the same tree. **Nothing is changed, nothing is dropped, nothing is added** — the only row in the table with a zero in every one of those three columns.

The ten it re-serialises are `word/document.xml`, `word/comments.xml`, `word/footnotes.xml`, `word/endnotes.xml`, both headers, both footers, `word/settings.xml` and `word/styles.xml` — the parts the SDK owns. The twenty-five it copies are everything else, including the **entire binary payload**: the chart *and its two style companions*, all four SmartArt parts, both embeddings, the image and the `altChunk` HTML.

This is the strongest row in the table, and it is strongest in the column that matters. Folio copies more bytes — 34 against 25 — but Folio *changes* `word/document.xml`. Docxodus changes nothing at all: it has the highest pass-through count of any participant that also has a zero in the "changed" column. It gets there without a policy, because the SDK owns the parts it models and passes the rest through entry-for-entry.

What it does not do. It orphans nothing, it dangles nothing (§5.4), and its content-types declarations are coherent (§5.5). Its relationship ids are the corpus's own and every one resolves.

And the interesting caveat is that it is not an editor. A user cannot restyle a chart through Docxodus's package writer, because the chart is a copied ZIP entry and a copied ZIP entry cannot be edited. The pass-through is *exact*, and it is exact because nobody looked at those bytes — which is §1.1's definition, arrived at by a completely different route.

6.8 Fidus Writer, canvas-editor, docs.plus, Plate.js — zero

All four regenerate everything. There is nothing to orphan and nothing to dangle, because there is nothing left of the author's package. **The failure mode of full regeneration is not a broken link; it is a document that is not the document the user opened.**

- **Fidus Writer** writes the *template's* package.
- **canvas-editor** writes the `docx` npm package's defaults, and re-emits the corpus's image under a **random new name** — `word/media/image1.png` becomes `word/media/zsakc18onfp21xzadj5t0.png`, byte-identical content under a new name (`docx@8.6.0`, `build/index.mjs:11191`, `:10332`). A part map that compared only *names* would score this as a drop plus an add.
- **docs.plus** writes its own default set and emits **no headers and footers at all**.
- **Plate.js** writes the smallest package in the trial: 11 parts, nothing added, nothing passed through.

---

7. What this means

- **Pass-through is a measure of what the editor ignored.** Folio's 34 byte-identical parts are 34 parts in which no user can ever make a change. The number is a fidelity number and an editing-power number at the same time, and it moves in opposite directions.
- **The strongest fidelity and the weakest editing are the same property.** An editor that copies a part byte-for-byte cannot round-trip a change to it. There is no configuration that gives you both, and no editor in this study tries.
- **"The part is still there" is not "the part is still used" — but on this corpus it did not happen.** The obvious orphan is the `altChunk`: a payload part in the package and an element in the body that points at it, in two separate places. **Seven participants keep both halves and six keep neither; not one keeps one.** The failure mode is real in general and it did happen with the chart (§5.4) — but it did not happen with the `altChunk`, and a reader who assumed the part map implied the orphan would have been wrong.
- **A dangling reference is worse than a dangling part, because it hides.** office-open and Docen produce packages with no missing parts, no dangling content-type declarations, no schema violations — and a chart and a SmartArt that are present, linked from the `.rels` file, and pointed at by body elements that resolve to nothing. **Every structural check passes. The page is still wrong.**
- **Check the forward direction.** Every relationship-integrity checker worth writing resolves `.rels` → parts. The direction that broke here is body → `.rels`. Five lines of code (`build/rel_forward.py`) find a defect that a 99-check fidelity score, a schema validator, and a content-types audit all miss.
- **The best row and the best score are not the same row.** Docxodus has the only all-zero row in the part map — 0 changed, 0 dropped, 0 added — and the *entire* binary payload byte-identical. Folio copies more bytes still (34 against 25), but it rewrites the body. If your requirement is "save it back and change nothing", Docxodus is the only participant in this study that satisfies it outright.
- **Choose on the axis you care about.** If you need the document to survive a round trip with its payloads intact, Folio and SuperDoc. If you need to *change* a chart, a diagram or an embedded workbook, none of the pass-through editors can help you and you want WordCraft, which redraws the payload from its own model instead. If you need a document Microsoft Word will open, note that **four of the fourteen outputs in this study are refused by Word 16.115** — Folio, office-open, Docen and Fidus Writer — for reasons set out in the companion report *Malformed Parts*. Two of those four pass every static check in this report.

Reproducing this

```
build/part_map.py                 # package-level serialization map
build/part_map.py --matrix        # the full part x editor matrix
build/part_matrix_md.py           # the matrix as four readable tables
build/part_map.py --detail        # per-part listing for one editor
build/orphan_audit.py             # unreferenced / body-orphaned / dangling parts
build/rel_forward.py              # body -> .rels resolution of every r:id
build/ct_coherence.py             # content-types declarations vs package contents
build/payload_audit.py            # payload parts and body references per editor
build/ns_map.py                   # per-namespace survival (the companion report)
build/make_report.sh <report.md>  # markdown -> .docx, with the two Word fixes
```

`build/make_report.sh` is the pipeline that produces this file: pandoc writes `w:dirty="true"` on the table-of-contents field, on which Word 16.115 asks whether to update the fields (the first report recorded it as a refusal; Report B §2), so `build/fix_dirty.py` rewrites it; and pandoc sizes every table for a portrait page, so `build/retune_tables.py` re-fits the tables to the landscape content width.

The per-editor code citations refer to the pinned commits listed in the first report, under `repos/`. **None of those checkouts was modified.**

Gaps and honest limits

1. **The compressed-stream claim about Folio comes from its source, not from a measurement.** `build/part_map.py` compares decompressed content, so the fact that Folio also preserves the original deflate stream and CRC is a statement about `docx-core/src/zip/archive.ts:388-405`, not a separate measurement of the output bytes.

2. **`w:altChunk` presence is detected by a literal string search** in `word/document.xml` (`"w:altChunk" in xml`), not by an XML query. It is sufficient for the corpus and the twelve outputs, all of which use the `w:` prefix for that element, but it is not prefix-independent.
3. **The `unreferenced`/`body-orphaned` distinction depends on how the fixture binds the chart's style and colour parts.** In the corpus they are bound by part-name convention and a content-types declaration rather than by a relationship, so they are "unreferenced" in the corpus as well as in every output that keeps them. That is a property of the format for those two parts, not a defect of any editor.
4. **The dangling-reference check resolves ids within the part that uses them.** A relationship id used in `word/document.xml` is resolved against `word/_rels/document.xml.rels` — correct — but the check does not follow the chain one level deeper (e.g. from the chart part into the embedded workbook's own rels). That deeper chain was checked separately by `orphan_audit.py`, and no output has a dangling part at any level.
5. **LibreOffice's row is a baseline, not a result.** It is not one of the twelve editors in the study and there is no source evidence to cite for its write path; its numbers here are measurements only.
6. **This report measures packages, not pages.** Whether a surviving part is *drawn* is a separate question, answered in *Rendering Versus Word*. For Docxodus the two answers are unusually far apart: its package keeps every namespace, and its renderer substitutes a `[SMARTART]` placeholder and a `[MATH]` chip for two of them.
7. **Docxodus's row is measured through the `DocxSession` API, not through its export CLI.** The round trip is `docxodus/core` → `openDocxSession` → `save()` (`harness/ui/docxodus/roundtrip.mjs`). The `@docxodus/export` CLI produces HTML and PDF and does not write DOCX, so it has no package to measure. The two paths share the same package writer, but the measurement exercises the session API — which is the product's own supported round-trip surface.
8. **Docxodus's pass-through is exact because the SDK never opens those parts.** "Byte-identical" here means *never touched*, and that is a statement about the SDK's part ownership, not about a decision to preserve. The consequence for an evaluator is the same one §4.1 draws for Folio: the parts Docxodus copies are the parts no user can change.